

How Many Iterations are Enough? Bayesian Outlier-Aware Sequential Testing for Conservative Randomized Verification

Richard Carson

May 11, 2026

Abstract

Determining the number of iterations required for randomized tests to reliably detect edge cases is a common challenge in software testing. Traditional approaches, such as fixed iteration counts or fuzzing either risk insufficient coverage or incur high computational costs, and are often unsuitable for continuous integration (CI) pipelines.

We introduce Bayesian Outlier-Aware Sequential Testing (BOAST), a method that combines sequential analysis with event-driven Bayesian inference to adaptively determine test duration based on observed failures and domain knowledge of the data input to the system under test. BOAST balances computational cost with confidence in test outcomes, remains robust under misspecified parameters, yields prior-independent test bias, and provides reliable 95% confidence intervals for the underlying failure rate. Empirical evaluation demonstrates strong performance under simulated independent normally distributed errors, suggesting suitability for CI and other resource-constrained testing environments. We also discuss the implications of our findings for broader software testing practices and potential avenues for future research.

1 Introduction

1.1 Problem Statement

1.1.1 Software Testing Perspective

When testing code consuming numerical data, it is common to use random transforms to produce synthetic data sets for testing.

However, this introduces the probability that any given run of the test will fail to catch edge cases, or fail due to a specific random seed.

The problem we address in this paper is: *How can we efficiently and reliably estimate the required number of test runs needed to establish the absence of failures with a certain confidence level, given an unknown relationship between the input data and test failure, and knowledge of the distribution of the input data?*

1.1.2 Mathematical Setup

Population and Sets

Each input to the function under test consists of a fixed-size set of elements $X^{(j)}$.

Let $P = \{X^{(1)}, X^{(2)}, \dots\}$ be a countably infinite population of sets, representing all possible inputs for the function under test F .

$$X^{(j)} = \{X_0^{(j)}, X_1^{(j)}, \dots, X_{n-1}^{(j)}\}, \quad j \in \mathbb{N}.$$

Each element $X_i^{(j)}$ is drawn i.i.d. and has probability p of being an **outlier** (an event we denote $O_i^{(j)} = 1$ if $X_i^{(j)}$ is an outlier, 0 otherwise):

$$\Pr[O_i^{(j)} = 1] = p$$

Where an **outlier** is defined as an element exhibiting characteristics associated with an increased likelihood of test failure.

Blackbox Function

There is a function under test, which we treat as a blackbox function

$$F : \mathcal{X} \rightarrow \{0, 1\}, \quad F(X^{(j)}) = \begin{cases} 1 & \text{if set passes} \\ 0 & \text{if set fails} \end{cases}$$

We assume there exists a relationship between the presence of outliers in ($X^{(j)}$) and failure, but the exact nature of this relationship is unknown:

$$\text{unknown mapping } F(X^{(j)}) \sim f(O_0^{(j)}, \dots, O_{n-1}^{(j)})$$

Sampling Problem

Let $(S \subseteq P)$ be the subset of sets actually observed/tested.

Define the event

$$E = \exists X^{(j)} \in S : F(X^{(j)}) = 0$$

i.e., "we observe at least one failing set."

The goal is:

Find the minimal $|S|$ such that $\Pr[E] \geq \gamma$

where $(\gamma \in (0,1))$ is a target confidence (e.g., 0.95).

Corresponding to the problem of finding the minimum number of samples needed to have a confidence of at least (γ) that we have observed at least one failing set, given the unknown relationship between outliers and failure, and the probability of outliers in the population.

1.2 Comparison to Other Methods

Below we outline existing approaches to the problem, and their limitations compared to the proposed BOAST method.

1.2.1 Common Approaches in Industry

This problem is commonly addressed in industry using heuristics such as fixed-budget testing, adversarial thinking, or fuzzing, depending on the specific context of the test and the resources available.

Fixed-budget testing

Approach Select a large fixed number of iterations k based on heuristics, and run the test for that many iterations [5].

Advantages This method is simple and easy to implement.

Limitations It does not provide any guarantees about the confidence level of detecting failure and as such can result in a number of iterations that is either too low (leading to false confidence in the reliability of the code), or too high (leading to wasted computational resources). Additionally, in the case of the absence of failure, this method provides no ability to stop early, and thus can waste resources.

Worst-case / adversarial thinking

Approach Design tests to specifically target edge cases and potential failure modes, rather than relying on random sampling.

Advantages Very effective for regression testing (i.e., testing for specific known issues) and can be more efficient than random sampling if the failure modes are well understood.

Limitations Cannot provide a comprehensive assessment of the code’s reliability, and can be time-consuming to design and implement. Additionally, is not effective in catching unknown failure modes or edge cases that were not anticipated during test design.

Fuzzing

Approach Generate random or semi-random inputs to test the code, with the goal of uncovering edge cases and potential failure modes.

Advantages Very comprehensive; can be effective in catching a wide range of issues, including those that were not anticipated during test design. Can also be designed with stopping criteria based on observed pass/fail ratios, which can provide some efficiency benefits compared to fixed-budget testing.

Limitations Computationally expensive, as it requires a large number of test iterations to achieve a high confidence level of detecting failure. Fuzzing is also not normally suitable for use in CI pipelines or other time-sensitive contexts, as it requires a large number of iterations. Additionally, it does not take into account domain-specific knowledge about the data or the code under test, which can lead to inefficient testing and missed failure modes.

1.2.2 SPRT (Sequential Probability Ratio Test)

Approach A statistical method for testing a hypothesis by evaluating the likelihood ratio of observed data under two competing hypotheses, and stopping when the ratio exceeds a certain threshold [7].

Advantages Provides a rigorous statistical framework for testing hypotheses and can be more efficient than fixed-budget testing. It also attempts to minimize the expected number of samples needed to reach a conclusion.

Limitations For noisy data, SPRT can theoretically require an infinite number of samples to reach a conclusion, as the likelihood ratio may never exceed the threshold. Additionally, SPRT does not take into account domain-specific knowledge about the data or the code under test, which can lead to inefficient testing and missed failure modes.

1.2.3 Recent Literature

An Efficient Adaptive Sequential Procedure for Simple Hypotheses with Expression for Finite Number of Applications of Less Effective Treatment

Approach Unlike SPRT, this method addresses the problem of drift in case of noisy data, reducing the probability of infinite sampling. It does so by introducing an adaptive procedure that adjusts the testing strategy based on observed data [4].

Advantages Just as rigorous as SPRT, while addressing the excessive sampling problem in the case of noisy data. Additionally it introduces a closed-form expression for the expected number of inferior allocations, allowing for precise estimation of the "cost of testing" in live environments.

Limitations Like SPRT, this method does not take into account domain-specific knowledge about the data or the code under test, which can lead to inefficient testing and missed failure modes. Additionally, while it addresses the issue of infinite sampling in noisy data, it may still require a large number of samples to reach a conclusion in cases where the data is particularly noisy or the difference between hypotheses is small, and as such may not be suitable for use in time-sensitive contexts such as CI pipelines.

Classifier-Based Nonparametric Sequential Hypothesis Testing

Approach A hypothesis testing method that uses a classifier to distinguish between two distributions, and applies sequential testing to determine which distribution is more likely [3].

Advantages Adaptable to complex data distributions, and claims to identify the ground-truth almost surely.

Limitations Requires training a classifier, which can be computationally expensive and often domain or application specific. Additionally, the method does not provide guarantees on the number of samples needed to reach a conclusion, and can be inefficient in cases where the distributions are similar. The use of a classifier also introduces computational overhead and potential for overfitting in the training process.

Conservative Software Reliability Assessments Using Collections of Bayesian Inference Problems

Building on the work of Salako and Muhammad [6], who demonstrate that conservative software reliability testing requires moving away from singular statistical frameworks in favor of mixed approaches, this paper proposes a hybrid Bayesian-frequentist approach to the problem

of conservative software reliability testing. While the work of Salako and Muhammad utilizes a collection of Bayesian methodologies, our method employs an intentionally conservative Bayesian component to inform the stop criterion of a sequential frequentist test, which we demonstrate to be effective in controlling computational cost while maintaining reliable estimates of failure probability.

2 Proposed Solution

2.1 Overview

To address the challenges outlined in Section 1, we introduce BOAST (Bayesian Outlier-Aware Sequential Testing), a method for efficiently estimating the minimal number of test iterations required to achieve a desired confidence level of detecting failure in code consuming numerical data, particularly while testing against synthetic data with known noise characteristics.

BOAST combines a modified Bayesian update procedure with a frequentist stopping criterion, and incorporates a damping factor to control conservatism in the initial estimate of failure probability. The method is highly tunable, has no systemic bias towards false positives or false negatives, and is robust to mis-specification of its input parameters, making it suitable for a wide range of use cases and risk tolerances.

The output of the test includes both a pass/fail result, and a confidence interval for the true failure rate among data sets, allowing users to make informed decisions about the reliability of their code and the potential risks associated with it. The method is designed to be computationally efficient, making it suitable for use in CI pipelines and other time-sensitive contexts.

2.2 Algorithm Details

The BOAST method consists of three main phases, depicted in Figure 1: the estimation phase, the iterative testing phase, and the output phase.

We begin by defining the following parameters and constants used throughout the algorithm:

- n The size of the data set under test.
- p The likelihood of any individual point being an outlier, chosen conservatively (under-estimation of the true risk).
- q The desired confidence that enough iterations have been performed to have observed a failure, if one was present.
- pass_ratio The required ratio of passes in the observed sample of iterations to consider the test a success.

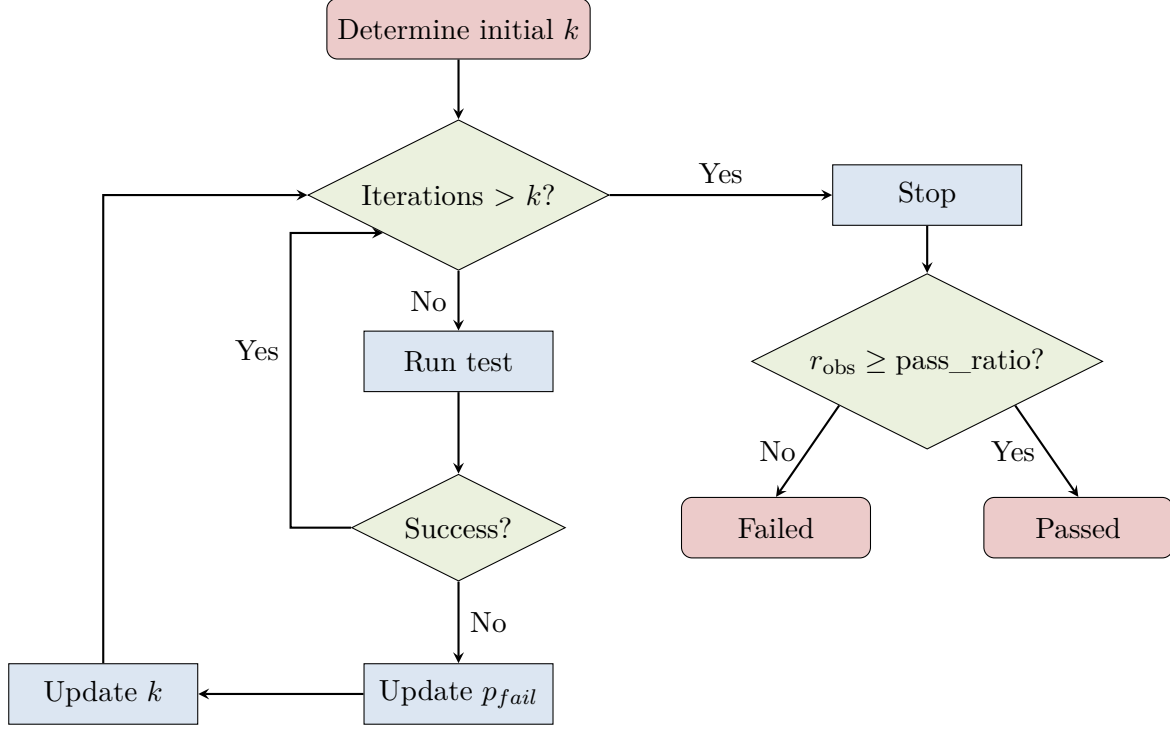


Figure 1: Flowchart of the BOAST testing procedure.

- C A damping constant reflecting the diminishing impact of additional data points on the likelihood of failure.
- p_s A strength parameter for the Bayesian prior, reflecting the confidence in the initial estimate of the failure probability.

Derivations for suggested values of the above constants are included in Section 2.4.

q represents the target confidence γ of observing a failing set within the sample population ($\Pr[E] \geq q$). It should be selected to be a reasonably conservative value, as defined in Section 2.4, with higher values indicating a lower tolerance for risk, and therefore a desire for more iterations to be performed.

p should be selected to be a conservative underestimate of the true point-wise probability of outliers, to ensure that the algorithm does not underestimate the required number of iterations. It can be thought of as the user's prior belief about the likelihood of any individual value in the input data being unusual or extreme enough to contribute to a failure in the code under test.

`pass_ratio` should be selected based on the user's tolerance for test failures, for example in the case of intermittent network failures, or the effects of non-determinism in the generated data sets. This parameter should be optional, defaulting to 1.0 (i.e., no failures observed in the sample), and can be used to allow for a certain level of observed failure in the sample while still considering the test a success, which may be desirable in cases where some level of failure is expected or acceptable.

C , the damping constant, controls the conservatism of the initial estimate for p_{fail} . Our recommended value is derived in Section 2.4.1 using a formulation based on the minimum number of iterations required for a test result to be considered statistically meaningful, and a value of confidence that is considered the minimum reasonably conservative value. Our recommended value for C is 1.864, which corresponds to a minimum number of iterations of 3 at a confidence level of 80%.

p_s , the strength of the Bayesian prior, is derived in Section 2.4.2 using a method that identifies a Pareto-optimal tradeoff between the accuracy of the test and the computational cost of the test procedure. Our recommended value for p_s is $3.35 \cdot 10^6$.

The sections below describe the 3-phase approach for the BOAST method.

2.2.1 Estimation Phase

Given n the number of items per set X , and p the point-wise lower-bound estimate for outlier frequency, we calculate an initial conservative estimate for the set-wise probability of failure p_{fail} :

$$p' = \frac{p}{(1 + C \cdot n \cdot p)}, p_{\text{fail}} = 1 - (1 - p')^n \quad (1)$$

p' is used to model the saturation of the effect of individual outliers on set-wise failure. It represents the inverse relationship $\frac{1}{np}$ between number of expected outliers, and their effects on iterations k . For more on the derivation of this form, and the recommended value for $C = 1.864$, see Section 2.4.1. We call the ratio applied to p the "damping fraction", since it serves to dampen the effect of increasing n on p_{fail} .

For small n the probability of failure is roughly linear in n , consistent with a binomial model where $p_{\text{fail}} \approx 1 - (1 - p)^n$. As n becomes large, the damping fraction saturates, and the effective failure probability approaches a limiting value $p_{\text{fail}} = 1 - e^{-1/C}$. In this regime, the number of failures per iteration can be approximated by a Poisson distribution with $\lambda = \frac{1}{C}$, reflecting the diminishing returns of increasing data set size on the likelihood of failure $F(X)$ due to the presence of outliers in the set.

An initial estimate for the number of iterations k required to achieve the desired confidence q of observing at least one failing set in the collected sample S (that is to say the likelihood estimate for $\Pr[E]$ given q) can then be computed as:

$$k = \left\lceil \frac{\ln(1 - q)}{\ln(1 - p_{\text{fail}})} \right\rceil \quad (2)$$

Where p_{fail} is the probability of any single data set failing the test.

Finally, Bayesian priors for p_{fail} are calculated using p_s :

$$\alpha = p_{\text{fail}} \cdot p_s, \quad \beta = (1 - p_{\text{fail}}) \cdot p_s \quad (3)$$

For more on the derivation of this form, and the recommended value for $p_s = 3.35 \cdot 10^6$, see Section 2.4.2.

2.2.2 Iterative Testing Phase

In this phase we will track the following variables:

k The current estimate for iterations required to achieve the given confidence

α, β The priors for our Bayesian updates

iterations The number of iterations performed so far

passes The number of successful iterations

passes__since__failure A tally of successes since the last failure

The test involves running a blackbox testing routine on a new set of size n , $X^i \in P$, in each iteration in order to evaluate $F(X^i)$. We recommend each test utilize a seeded transform on an initial input set.

For each iteration, perform the following steps:

Stopping Criteria

The current value for k represents the number of iterations required to have confidence q of having observed at least one failure, given the current estimate for set-wise failure probability p_{fail} .

Thus, if $\text{iterations} \geq k$, we can be confident that we would have observed at least one failure if the true failure probability is equal to or greater than our current estimate, and thus we can stop the test and move on to the output phase.

Timeout Criteria

To prevent unreasonable runtimes in the case of extremely low failure probabilities in the absence of observed failure, we recommend implementing a timeout mechanism. If the total elapsed time for the testing process exceeds a predetermined threshold, the test should terminate and report failure due to insufficient evidence to support a claim.

Update Procedure

Note:

With our assumptions that p_{fail} represents a conservative underestimate, and that at least one failing set exists, we can attribute any observed successes prior to a failure to chance, and therefore the update procedure should only be performed on failure, since this is the only time we can be sure that we have observed a real failure rather than just random chance. Additionally, this prevents k from growing in the base-case where no failures exist.

- Increment *iterations* by 1 to reflect the new test iteration, and perform the blackbox test on the new set. The test should determine if the set passes or fails.

- If the set passes, we increment *passes* and *passes_since_failure* by 1, and repeat the procedure for the next iteration.
- If the set fails, we update our estimates:
 - Increment *alpha* by 1, to represent the detected failure
 - Increment β by *passes_since_failure*, and reset *passes_since_failure* = 0
 - Recalculate the estimated set-wise failure risk: $p_{\text{fail}} = \frac{\alpha}{\alpha + \beta}$
 - Update the number of iterations required: $k = \left\lceil \frac{\ln(1-q)}{\ln(1-p_{\text{fail}})} \right\rceil$
 - Repeat the iteration procedure

2.2.3 Output Phase

Once the required evidence has been collected, we can calculate the observed ratio of passes to failures r_{obs} as:

$$r_{\text{obs}} = \frac{\text{total passes}}{\text{total iterations}} \quad (4)$$

The final test result can then be determined by comparing r_{obs} to the user-defined *pass_ratio*:

$$\text{test result} = \begin{cases} \text{pass} & \text{if } r_{\text{obs}} \geq \text{pass_ratio} \\ \text{fail} & \text{if } r_{\text{obs}} < \text{pass_ratio} \end{cases}$$

In order to provide a robust estimate of the true underlying failure rate amongst data sets from the sample collected by BOAST, we construct a 95% Wilson score confidence interval [8]. This method is chosen for its superior performance compared to other methods such as the normal approximation interval, particularly in cases of small sample sizes or extreme proportions, which are common in our testing context.

The interval bounds are calculated as follows:

$$p_{\text{fail, lower/upper}} = \frac{1}{1 + \frac{z^2}{k}} \left[\hat{p} + \frac{z^2}{2k} \pm z \sqrt{\frac{\hat{p}(1 - \hat{p})}{k} + \frac{z^2}{4k^2}} \right] \quad (5)$$

where:

- $\hat{p} = 1 - \text{pass_ratio}$ is the observed failure proportion.
- k is the total number of iterations (sample size).
- z is the $1 - \frac{\alpha}{2}$ quantile of the standard normal distribution (e.g., 1.96 for 95% confidence).

This method cleanly separates the Beta function used to estimate the required size of the sample k from the analysis of the observed sample, which allows for a more robust estimate of the true underlying failure rate among data sets given the purposely conservative nature of the initial estimate for p_{fail} .

The outputs for the test are then:

- test result The test result
- r_{obs} , the observed pass ratio
- The calculated 95th percentile bounds on $p_{\text{fail, true}}$

These can be communicated succinctly as: “[$100 \cdot r_{\text{obs}}$]% of tests passed. With 95% confidence, the true failure rate is between [$100 \cdot p_{\text{fail, lower}}$]% and [$100 \cdot p_{\text{fail, upper}}$].”

2.3 Limitations

The proposed method has several limitations and assumptions worth noting:

- The size of the data set n must be known in advance, and should remain constant throughout the testing process.
- The method assumes that the probability of outliers is independent and normally distributed across the data set, which may not hold true in all cases.
- Very small n (< 100) require a very large number of iterations to converge; this is an expected by-product of the model used, and we therefore recommend $n \geq 100$

2.4 Derivations

2.4.1 The Damping Constant - C

As n increases, the naive binomial form $p_{\text{fail}} = 1 - (1 - p')^n$ for expected failure trends to 100%. To resolve this, we introduce the damping fraction $p' = \frac{p}{(1+C \cdot n \cdot p)}$

We began with the form

$$p' = \frac{p}{n \cdot p} \tag{6}$$

In order to eliminate the undefined case at $n = 0$, and to ensure that $p' \leq p$ in all cases, we modified this to the form:

$$p' = \frac{p}{(1 + n \cdot p)} \tag{7}$$

And finally introduced the constant C to allow for tuning of the saturation effect:

$$p' = \frac{p}{(1 + C \cdot n \cdot p)} \quad (8)$$

C represents the binding strength modulating between outliers, and thus the rate of saturation of the effect of increasing n on p_{fail} .

The effect is the introduction of a asymptotic floor on the minimum number of iterations required for a set as $n \rightarrow \infty$, k_{floor} .

To find a reasonable value for C , we define the reasonable universe of parameters for the test space. For a theoretical set of infinite size, there is naturally a minimum number of iterations under which the test is not useful - k_{floor} . There is also a practical maximum number, above which the computation required for extremely large sets becomes infeasible.

We will define that practical range for k_{floor} to be between 3 and 10 iterations, and then we will define the lowest confidence q we consider reasonably conservative to be 0.8.

The practical lower bound for k_{floor} is defined by the lowest number of iterations that could be considered valid in a test such as the Rule of Three, where $k < 3$ would produce a confidence level above 100% for any observed pass ratio, and thus would not be useful for any real-world use case.

We can analyze the effect of C effect on k_{floor} as n approaches infinity:

$$p' = \frac{p}{1 + Cnp} \quad (9)$$

$$p_{\text{fail}} = 1 - (1 - p')^n \quad (10)$$

For large n , $(1 - p')^n \rightarrow e^{-np'}$, so

$$p_{\text{fail}} = 1 - e^{-\frac{np}{1+Cnp}} = 1 - e^{-\frac{1}{C} \cdot \frac{Cnp}{1+Cnp}} \quad (11)$$

As $n \rightarrow \infty$, $\frac{Cnp}{1+Cnp} \rightarrow 1$, giving us $p_{\text{fail}} = 1 - e^{-1/C}$

Then

$$k = \frac{\ln(1 - q)}{\ln(1 - p_{\text{fail}})} \Rightarrow k_{\text{floor}} = \frac{\ln(1 - q)}{\ln(e^{-1/C})} = \frac{\ln(1 - q)}{-1/C} = -C \ln(1 - q) \quad (12)$$

Solving for C :

$$C = \frac{k_{\text{floor}}}{-\ln(1 - q)} \quad (13)$$

Since $-\ln(1 - q)$ represents the cumulative hazard function, we can interpret C in units of iterations / *nat*, or the average number of iterations required to resolve 1*nat* of uncertainty.

If we set $k_{\text{floor}} = 3$ as a reasonable minimum number of iterations that could be considered useful, given a lower-bound on the value for q we consider reasonably conservative of $q = 0.8$, we find:

$$C = \frac{k_{\text{floor}}}{-\ln(1 - q)} \Rightarrow C = \frac{3}{-\ln(1 - 0.8)} \approx 1.864$$

Additionally, since

$$k_{\text{floor}} = -C \cdot \ln(1 - q),$$

we can evaluate the sensitivity of k_{floor} to small variations in C for a fixed confidence level of $q = 0.8$.

Using the derived nominal value $C = 1.864$:

$$k_{\text{floor}} = -(1.864) \ln(1 - 0.8) \approx 2.99999$$

Reducing C by 0.5:

$$k_{\text{floor}} = -(1.864 - 0.5) \ln(1 - 0.8) \approx 2.19527$$

Increasing C by 0.5:

$$k_{\text{floor}} = -(1.864 + 0.5) \ln(1 - 0.8) \approx 3.80471$$

Taking into account the effect of the ceiling operation in the calculation of k_{initial} , variations in C of approximately ± 0.5 around the nominal value of 1.864 produce a change of approximately ± 0.804 in k_{floor} , indicating that the formulation is reasonably robust to small perturbations in C .

We will therefore recommend $C = 1.864$ as the nominal value for the damping constant. Since 1 nat represents ≈ 1.44 bits of information, this corresponds to an average of 1.292 iterations per bit of information gained about the failure probability.

To verify this value, we generated the k_{floor} produced by a variety of values within the defined parameters space, and then took C_{mean} and C_{σ} to produce the table and graph shown in Figure 5.

From the table we can see that the $C \approx 1.8$ produces values right in our defined practical range for k_{floor} across a variety of confidence levels, confirming our derived value.

If a use-case requires a different practical range for k_{floor} or q , a value for C should be selected using the formula above (13).

We recommend selecting a new value if the scale of compute available for regular testing is significantly above what is standard. k_{floor} should then be selected to reflect the smallest number of iterations that would have statistical meaning in context. q should be selected to reflect the confidence in that floor, that is the resistance to growing k_{floor} as q increases.

2.4.2 Prior Strength - p_s

Isolating the effects of parameters on bias

Our first step to determining a reasonable p_s for a given model was to run the experiment described in Section 3.5, varying p_{fail} and pass_ratio over the base-case of $p_s = 1$, to determine if either parameter had a significant effect on bias.

These parameters were chosen since they are calculated directly from n and p , and thus allow us to infer the effect of those parameters on bias.

For this and all experiments in this section we define bias as the difference between the test's observed pass ratio r_{obs} , and the true underlying pass rate for the simulated trial, r_{true}

$$\text{bias} = r_{\text{obs}} - r_{\text{true}} \quad (14)$$

A positive bias indicates the test overestimated the pass rate (false negative), while a negative bias indicates the test underestimated the pass rate (false positive).

Our hypothesis was that the delta between the expected level of failure p_{fail} and the acceptable level of failure $1 - r_{\text{pass}}$ would be a good predictor of bias.

However, as can be seen in Figure 6, neither parameter had a measurable effect on bias - which appeared to be randomly distributed around 0 with a constant error regardless of the values of p_{fail} and pass_ratio . Since p_{fail} encodes for k , and contains both n and p , we can infer that none of n , p , p_{fail} , or k have a significant effect on bias, which appears to be independent of any inputs into the model given an unknown underlying failure rate.

Effect of prior strength on bias and compute

Our next step was to run a simplified experiment varying only p_s , shown in Figure 8a (a), to determine the effect of p_s on bias. This demonstrated the bias was indeed centered at 0, with a variance that decreased with increasing p_s , plateauing near $p_s = 1e7$.

We next ran the same experiment measuring the fraction of initially estimated iterations required to converge at various values of p_s , shown in Figure 8b (b). This demonstrated that increasing p_s also increased the required computation, again reaching an asymptotic limit, this time around $p_s = 1e8$.

Modeling the tradeoff between bias and compute

Following the experiment in Section 2.4.2, we fit the relationship between the performed fraction of maximum iterations and p_s to a pair of Hill equations [2] of the form:

$$y = y_{\text{min}} + (y_{\text{max}} - y_{\text{min}}) \frac{x^n}{k^n + x^n} \quad (15)$$

Where:

- y is the observed variable (bias stddev or compute fraction)

- x is p_s , the prior strength
- n is the Hill coefficient, which determines the steepness of the curve
- k is the value of x at which y reaches half of its maximum value
- y_{min} and y_{max} are the starting and ending values of y as x increases

Bias was first normalized to the maximum observed value, and compute was already a fraction of the maximum, making both variables suitable for fitting to this form.

The hill curve was chosen due to the need for a monotonic, saturating sigmoid function.

This resulted in a good fit for both bias and compute ($R^2 > 0.998$ for both curves).

For the curve of bias against p_s , we found the following:

$$y = 1.0 + (0.323 - 1.0) \frac{x^{0.548}}{(4.77 \cdot 10^4)^{0.548} + x^{0.548}}$$

For the curve of compute against p_s , we found the following:

$$y = 0.111 + (1.0 - 0.111) \frac{x^{0.639}}{(6.70 \cdot 10^5)^{0.639} + x^{0.639}}$$

As expected, n , or the steepness of the curve, was below 1 for both curves, indicating a gradual slope towards a saturation point, rather than a sharp cliff. The k values indicate that the bias curve reaches half of its maximum value at a much lower p_s than the compute curve, indicating that bias is more sensitive to changes in p_s at lower values, while compute continues to degrade significantly as p_s increases. This results in a "window of efficiency" in which bias is relatively insensitive to changes in p_s , while compute continues to improve significantly, which we can use to select a recommended value for p_s .

Selecting a recommended prior strength

We then defined a Lagrangian objective function [1] to balance the competing goals of minimizing bias and minimizing compute, introducing β , the tradeoff coefficient between the two objectives representing the relative importance of compute savings versus bias reduction:

$$\text{objective_score}(p_s) = \text{bias_std}(p_s) - \beta \cdot \text{compute_frac}(p_s) \quad (16)$$

By sweeping β over logarithmic space from 10^{-5} to 10^1 , or in other words, from a strong preference for minimizing bias to an equal weighting of both objectives, and solving for the root of the objective function, we can build a table of optimal p_s values for various tradeoff preferences.

The result, shown in Figure 10, demonstrates a flat region of optimal p_s values centered at $3.35 \cdot 10^6$, in which bias becomes relatively insensitive to p_s , while compute savings continue

to improve significantly as p_s decreases. The tradeoff curve and suggested optimal value are shown in Figure 9.

We therefore recommend a nominal value of $p_s = 3.35 \cdot 10^6 \pm 1.98 \cdot 10^4$ for a good balance between compute and bias. Compared to the largest reasonable value of $p_s = 1e7$, this value results in only a 3% increase in bias stddev, while saving 24% of worst-case compute. By comparison, the less conservative value of $p_s = 4.43e5$ results in a 12% increase in bias stddev, while saving 39% of worst-case compute.

The recommendation corresponds to $\beta = 0.51$, meaning that compute savings are considered roughly half as important as bias reduction.

It is noteworthy that since compute is a fraction of the initially estimated iterations actually completed, this will already scale with n ; The larger the set size, the greater the potential savings as compared to the initially estimated worst-case compute. Thus, this recommended value for p_s should be robust across a wide range of n values.

3 Methodology

3.1 Experiment 1: Conservative Estimate of p_{fail} vs Simulated True p_{fail}

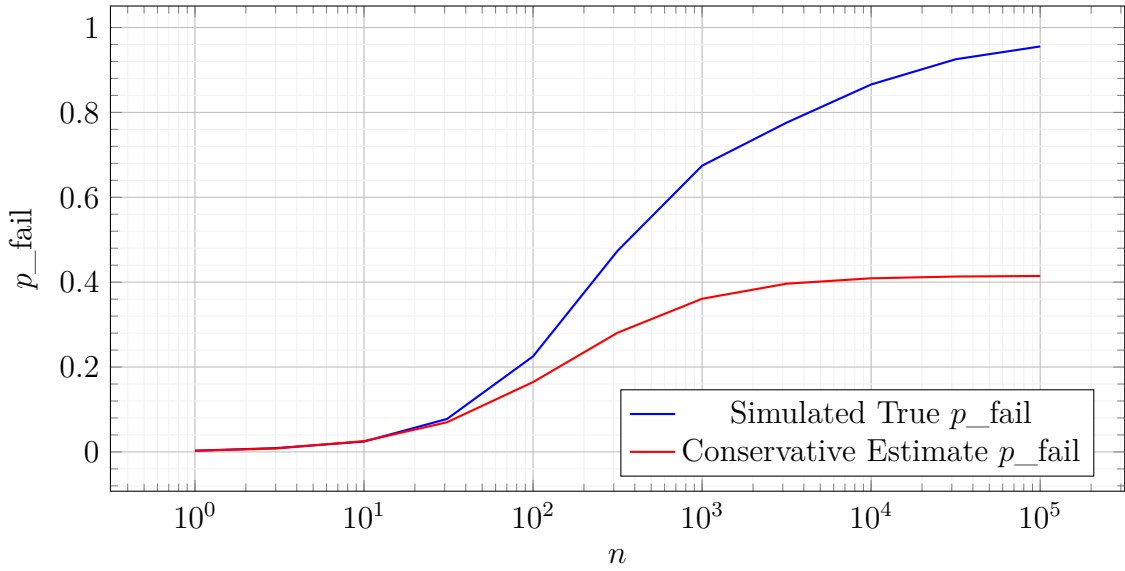


Figure 2: Graph showing the conservative estimates of p_{fail} vs simulated true p_{fail} for various dataset sizes n

Objective. To verify that calculated p_{fail} is a conservative underestimate of the true simulated p_{fail} across a range of dataset sizes n according to the outlier-aware model of the initial p_{fail} estimate in BOAST.

3.1.1 Setup.

- Vary set size n logarithmically from $1e0$ to $1e5$.
- For each n , run 10,000 trials.
- For each trial, simulate failure due to outliers, accounting for interactions between outliers:
 - Draw n samples from a standard normal distribution ($\mu = 0, \sigma = 1$).
 - Define outliers as samples with absolute value greater than 3.
 - A trial fails if the sum of all outliers exceeds 3 in absolute value.
- Record the observed failure rate across trials as the simulated true p_{fail} .
- For each n , calculate the corresponding conservative initial estimate of p_{fail} using the BOAST method:
 - Confidence level: 0.99
 - Outlier probability: 0.0027 (approximate probability of $|x| > 3\sigma$ in a normal distribution)
 - Pass ratio: 0.0 (not used in this experiment)

3.1.2 Definition of Metrics.

- $p_{\text{fail, true}}$: Observed failure rate from simulations.
- $p_{\text{fail, calc}}$: Calculated conservative estimate of p_{fail} using BOAST.

3.1.3 Hypothesis.

BOAST is expected to yield a calculated $p_{\text{fail, calc}}$ that is less than or equal to the observed $p_{\text{fail, true}}$ across all tested set sizes n .

3.1.4 Results.

As shown in Figure 2, $p_{\text{fail, calc}}$ consistently underestimates $p_{\text{fail, true}}$ across the full range of dataset sizes n . Moreover, the two curves exhibit similar functional form, indicating that the estimator preserves the underlying dependence on n while remaining systematically conservative.

3.1.5 Analysis / Interpretation.

The results confirm the hypothesis that the BOAST-calculated p_{fail} is a conservative underestimate of the true simulated p_{fail} across all dataset sizes tested.

By following the same trend as the true p_{fail} , the BOAST estimate avoids the trap of underestimating by enough to make initial k too large to be useful, as n increases, while still providing a reliable conservative estimate.

3.1.6 Summary / Recommendation.

The BOAST method provides a reliable conservative estimate of p_{fail} , without trend inversion as n varies, making it suitable for this application.

3.2 Experiment 2: Predicted k to Observed k to Failure

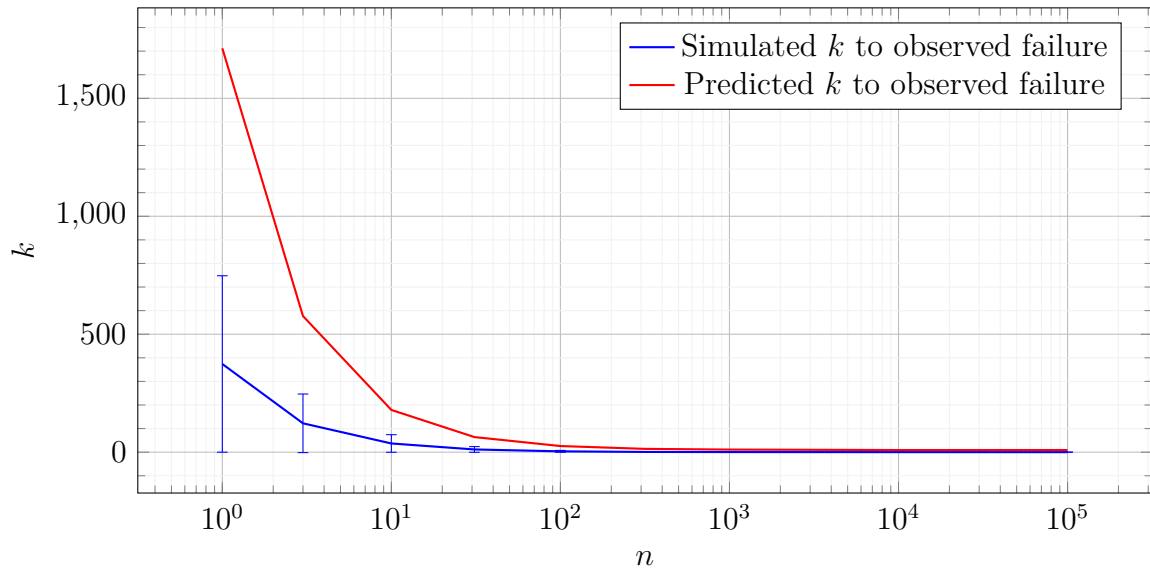


Figure 3: Graph showing the predicted number of iterations k to observed failure vs the simulated results

Objective. To verify that the predicted number of iterations k to failure from BOAST encompasses the observed number of iterations k to failure across a range of dataset sizes n according to the outlier-aware model of the initial p_{fail} estimate in BOAST.

3.2.1 Setup.

- Vary set size n logarithmically from $1e0$ to $1e5$.

- For each n , run 10,000 trials.
 - Draw n samples from a standard normal distribution ($\mu = 0, \sigma = 1$).
 - Define outliers as samples with absolute value greater than 3.
 - A trial fails if the sum of all outliers exceeds 3 in absolute value.
- For each trial, record the number of iterations k taken to find a failing set
- Take the mean and standard deviation of observed k across trials.
- After all trials for a given n , calculate the predicted k to failure using the BOAST method:
 - Confidence level: 0.99
 - Outlier probability: 0.0027 (approximate probability of $|x| > 3\sigma$ in a normal distribution)
 - Pass ratio: 0.0 (not used in this experiment)

3.2.2 Definition of Metrics.

- $k_{\text{obs, mean}}$: Mean observed iterations to failure from simulations.
- $k_{\text{obs, stdev}}$: Standard deviation of observed iterations to failure from simulations.
- $k_{\text{predicted}}$: Predicted iterations to failure using BOAST.

3.2.3 Hypothesis.

The BOAST is expected to yield a predicted $k_{\text{predicted}}$ that is greater than or equal to the mean observed $k_{\text{obs, mean}}$ across all tested set sizes n .

3.2.4 Results.

As shown in Figure 3, $k_{\text{predicted}}$ consistently overestimates $k_{\text{obs, mean}}$ across the full range of dataset sizes n . The predicted values remain above the observed means, following the same trend as observed k to failure, similarly to Experiment 1.

The observed standard deviation $k_{\text{obs, stdev}}$ shrinks as n increases, indicating more consistent behavior in larger datasets.

3.2.5 Analysis / Interpretation.

The results confirm the hypothesis that the BOAST-predicted k to failure is a conservative overestimate of the mean observed k to failure across all dataset sizes tested.

This means that for a well specified test, BOAST’s predicted k will likely include a failure within that number of iterations, providing confidence in the method’s reliability.

3.2.6 Summary / Recommendation.

The BOAST method provides a reliable conservative prediction of iterations to failure, making it suitable for this application.

3.3 Experiment 3: Confidence Interval on p_{fail} Estimates

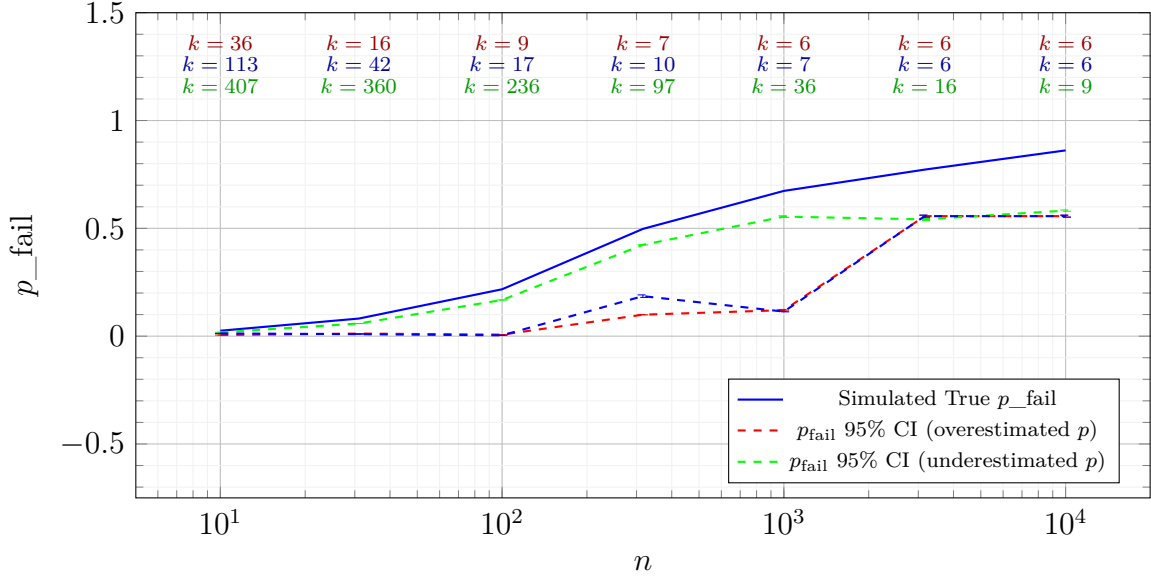


Figure 4: Graph showing the mean and standard deviation of lower-bound confidence intervals on the sample of iterations observed by BOAST from a simulation of 10,000 trials, compared to the simulated true p_{fail} for various dataset sizes n , and for various values of point-wise p . Green was deliberately underestimated, blue was exact, and red was deliberately overestimated. Markers indicate the number of iterations k performed by BOAST at each point.

Objective. To verify that the calculated p_{fail} from BOAST provides accurate confidence intervals that encompass the observed p_{fail} across a range of dataset sizes n and point-wise p values.

3.3.1 Setup.

- Vary set size n logarithmically from $1e0$ to $1e5$.
- Test 10,000 sets for each combination of n , to calculate observed p_{fail} .
- Run BOAST with the same results to calculate confidence intervals on p_{fail} for each value of point-wise p :
 - Underestimate of p : $p_{\text{outlier}} = 1e - 4$
 - Exact p : $p_{\text{outlier}} = 0.0027$ (approximate probability of $|x| > 3\sigma$ in a normal distribution)
 - Overestimate of p : $p_{\text{outlier}} = 0.01$

3.3.2 Definition of Metrics.

- p_{fail} : True failure rate from simulations.
- $p_{\text{fail, boast, min}}$: Lower bound of BOAST-calculated confidence interval for p_{fail} .
- $p_{\text{fail, boast, max}}$: Upper bound of BOAST-calculated confidence interval for p_{fail} .
- $p_{\text{fail, obs}}$: Observed failure rate in the sample BOAST selects.

3.3.3 Hypothesis.

The BOAST-calculated confidence intervals for p_{fail} are expected to encompass the observed p_{fail} for accurate and underestimated point-wise p values, and to potentially miss for overestimated point-wise p values, across all tested set sizes n .

3.3.4 Results.

As shown in Figure 4, the calculated confidence intervals for p_{fail} from BOAST successfully encompass the observed p_{fail} for both all tested set sizes n even for badly specified point-wise p values.

When point-wise p is underestimated (green), the confidence intervals are narrower and more accurate, closely tracking the observed p_{fail} .

When point-wise p is exact (blue), the confidence intervals remain accurate, but is much wider than the underestimated case.

When point-wise p is overestimated (red), the confidence intervals are significantly wider for small n and converge to the exact case as n increases, but still encompass the observed p_{fail} .

3.3.5 Analysis / Interpretation.

These results show that BOAST is robust to badly specified point-wise p parameters, but gains significant accuracy when point-wise p is well specified or conservative (i.e., underestimated).

An overly conservative (underestimated) point-wise p leads to tighter confidence intervals, due to the larger number of iterations BOAST performs in this case, improving the accuracy of the p_{fail} estimate.

An overly aggressive (overestimated) point-wise p leads to wider confidence intervals, especially for small n , due to the smaller number of iterations BOAST performs in this case, reducing the accuracy of the p_{fail} estimate. However, as n increases, the effect of point-wise p mis-specification diminishes, and the confidence intervals converge towards those obtained with exact point-wise p .

3.3.6 Summary / Recommendation.

The BOAST method provides reliable confidence intervals for p_{fail} estimates, particularly when point-wise p is well specified or underestimated, but is still useful when point-wise p is overestimated.

3.4 Experiment 4: Damping Constant Effect on k floor

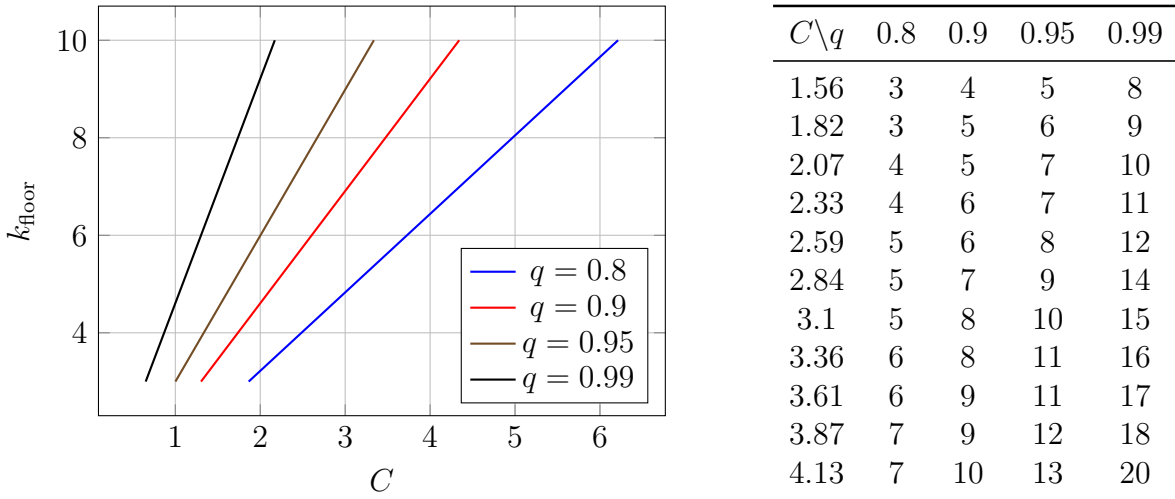


Figure 5: Left: Graph of k_{floor} vs C values in the defined parameter space. Right: Table showing values of k_{floor} for $C_{\text{mean}} \pm C_{\sigma}$ for various confidence levels q .

Objective. To investigate the relationship between the damping constant C , desired k floor and confidence level q .

3.4.1 Setup.

The experiment will be performed in 2 parts:

Part 1 Vary q smoothly from 0.8 to 0.99 in increments of 0.01. For each q , vary k from 3 to 10 in increments of 1. Compute the corresponding C value for each (k, q) pair using the formula provided below:

$$C = \frac{k}{-\ln(1 - q)}$$

This will provide a range of C values corresponding to the desired k floor and q values defined in the parameter space.

Part 2 Use the results from Part 1 to compute the mean C_{mean} and standard deviation C_{σ} of the damping constant C across all (k, q) pairs. For the range $C = C_{\text{mean}} \pm C_{\sigma}$, compute the corresponding k_{floor} values for specific confidence levels $q = [0.8, 0.9, 0.95, 0.99]$:

$$k_{\text{floor}} = C \cdot -\ln(1 - q)$$

This will provide a table of k_{floor} values for the defined range of C and specific confidence levels.

3.4.2 Definition of Metrics.

Damping constant C for each desired k floor and pass probability q .

3.4.3 Hypothesis.

We expect C to increase monotonically with k and q .

3.4.4 Results.

The results, shown in Figure 5, confirm that C increases monotonically with both desired k floor and pass probability q , ranging from < 1 to > 6 in our defined parameter space.

Narrowing the range to $C = C_{\text{mean}} \pm C_{\sigma}$ gives us a range of 1.56 to 4.13, where k_{floor} values between 3 and 10 correspond to C values between 1.82 and 2.07 for confidence levels between 0.8 and 0.99.

3.4.5 Analysis / Interpretation.

The results confirm the hypothesis that C increases monotonically with both desired k floor and pass probability, and gives us a practical basis to verify the value found analytically in Section 2.4.1.

3.4.6 Summary / Recommendation.

From the table, a value between $C = 1.82$ and $C = 2.07$ best represents the desired k floor range of 3-10 for confidence levels between 0.8 and 0.99.

3.5 Experiment 5: Bias vs p_{fail} and ratio_pass

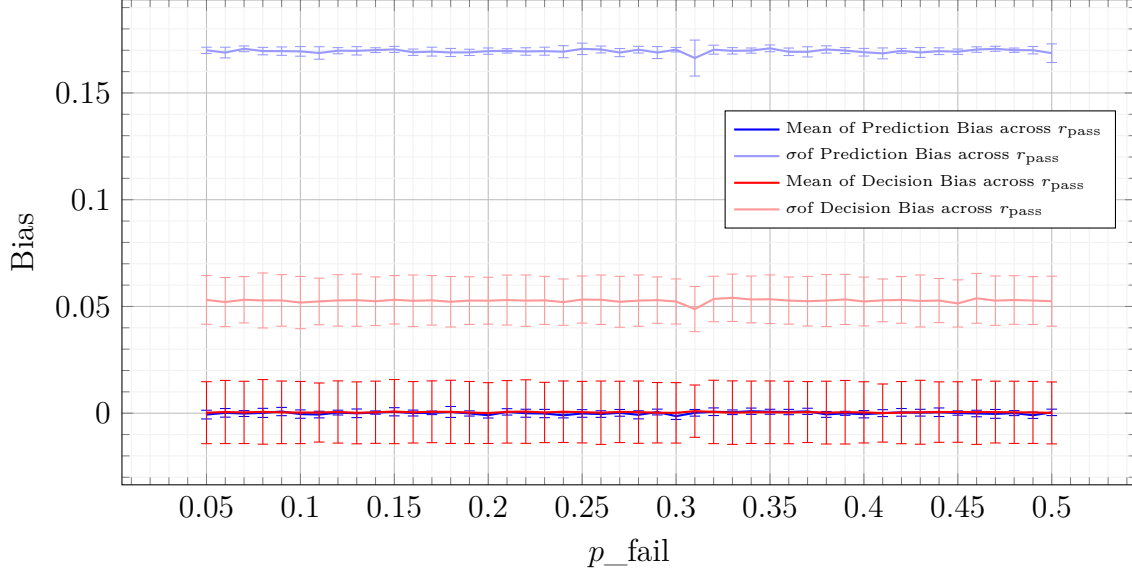


Figure 6: Graph demonstrating the lack of correlation between p_{fail} , pass_ratio , and bias in the base case of $p_s = 1$

Objective. To investigate the relationship between bias, p_{fail} , and pass ratio. p_{fail} is calculated using p , q and n , and so between it and pass_ratio we can explore the bias behavior of BOAST across the parameter space in the base-case of $p_s = 1$.

3.5.1 Setup.

- We select $\text{pass_ratio} \in \{0.01, 0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9, 0.99\}$.
- We vary p_{fail} from 0.05 to 0.50 in increments of 0.01
- For each p_{fail} , we compute the corresponding dataset size n using:

$$n = \left\lceil \frac{-\ln(1 - p_{\text{fail}})}{p'} \right\rceil, \quad p' = \frac{1}{1 + C \cdot n \cdot p}, \quad C = 1.864, \quad p = 10^{-6}$$

For each $(p_{\text{fail}}, \text{ratio_pass})$ pair, perform the 10,000 iterations of the following:

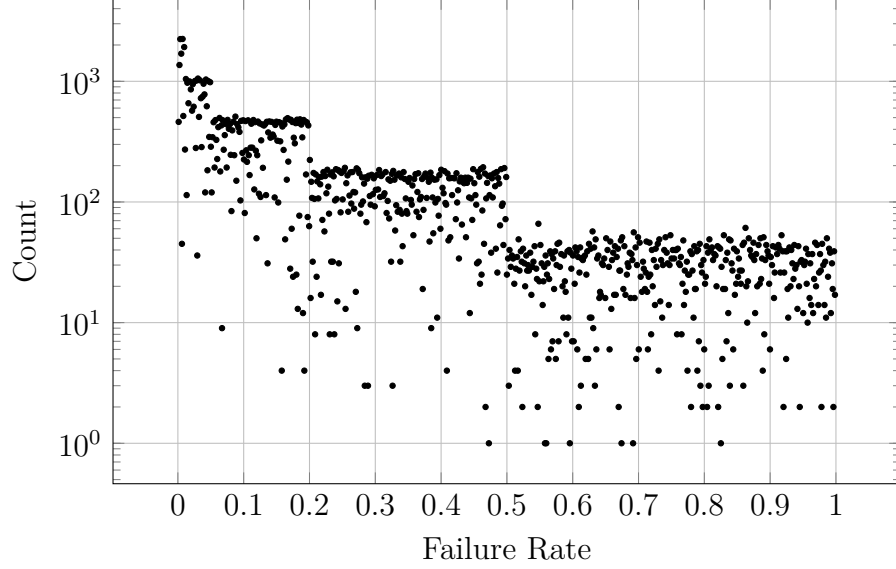


Figure 7: Scatter plot showing the distribution of true p_{fail} values across all iterations of the experiment, demonstrating the coverage of the parameter space.

- Draw a value x from a uniform distribution $U(0, 1)$.
- Determine range for the rate of failure based on the following piecewise function:

$$\text{failure_rate} = \begin{cases} [0.001, 0.01] & x < 0.1 \\ [0.01, 0.05] & 0.1 \leq x < 0.3 \\ [0.05, 0.20] & 0.3 \leq x < 0.65 \\ [0.20, 0.50] & 0.65 \leq x < 0.9 \\ [0.50, 0.999] & x \geq 0.9 \end{cases}$$

- Draw a true failure probability $p_{\text{fail, true}}$ from a uniform distribution over the determined range.
- Run BOAST using the failure rate found, noting the $p_{\text{fail, obs}}$ value at the end of the run, the initial estimate for k and the number of iterations actually performed.
- Compute prediction bias as $p_{\text{fail, obs}} - p_{\text{fail, true}}$.
- Compute the distance between pass_ratio and $p_{\text{fail, true}}$, and record it for use in the analysis
- Compute decision bias as 0 if the test result matches the expected result based on pass_ratio and $p_{\text{fail, true}}$, and the above distance if the test result does not match the expected result. This measures the quality of the final pass/fail decision made by BOAST.
- Compute the mean and standard deviation of prediction bias and decision bias across all iterations for each $p_{\text{fail, obs}}$ value.

- Compute the mean and standard deviation of the distance between pass_ratio and $p_{\text{fail, true}}$ across all iterations for use in the analysis of decision bias.

For each p_{fail} , and for both prediction bias and decision bias, compute:

- The mean of all bias means across all $p_{\text{fail, obs}}$ values.
- The standard deviation of all bias means across all $p_{\text{fail, obs}}$ values.
- The mean of all bias standard deviations across all $p_{\text{fail, obs}}$ values.
- The standard deviation of all bias standard deviations across all $p_{\text{fail, obs}}$ values.

3.5.2 Definition of Metrics.

- $p_{\text{fail, true}}$: True failure probability drawn from the defined piecewise distribution, magnifying the presence of segments of the parameter space that are more likely to be encountered in practice, while still covering the full range of p_{fail} values.
- p_{fail} : Calculated failure probability estimate from BOAST.
- $p_{\text{fail, obs}}$: Observed failure rate in the collected sample.
- pass_ratio : Pass ratio parameter provided to BOAST.
- Prediction bias: $p_{\text{fail, obs}} - p_{\text{fail, true}}$. This measures the quality of the sample collected by BOAST in terms of how well it estimates the true failure probability.
- Decision bias: $\text{pass_ratio} - p_{\text{fail, true}}$ if the final result is neither a false positive nor a false negative, 0 otherwise. This measures the quality of the final pass/fail decision made by BOAST.

3.5.3 Hypothesis.

We expect prediction bias, or the quality of the collected sample, to be a function of $p_{\text{fail, obs}}$, with bias increasing as $p_{\text{fail, obs}}$ increases, due to the increased difficulty of collecting a representative sample as failure becomes more likely. We also expect decision bias, or the quality of the final pass/fail decision, to be a function of the difference between pass_ratio and the hidden variable $p_{\text{fail, true}}$, with bias increasing as the difference narrows, due to the increased likelihood of making an incorrect decision as the true failure probability approaches the pass ratio threshold.

3.5.4 Results.

Contrary to our hypothesis, we find that as can be observed in Figure 6 prediction bias remains relatively constant across the parameter space, with mean bias values clustering around 0.0 and standard deviation of bias remaining low for all values of $p_{\text{fail, obs}}$. Both the mean and standard deviation of decision bias also remain relatively constant with a very small error, indicating that independent of p_s , the quality of the sample collected by BOAST is independent of the inputs to the model.

For decision bias, we find that similar to prediction bias, the mean bias values cluster around 0.0 with constant variance, while the standard deviation of bias is constant across the parameter space involved in p_{fail} and pass_ratio .

While the variance of the mean and standard deviation of decision bias is larger than that of prediction bias, using the raw distance $\text{pass_ratio} - p_{\text{fail, true}}$ in the absence of the piecewise function used for decision bias calculation, that difference is erased, with $\sigma_{\text{bias, decision}}$ matching $\sigma_{\text{bias, prediction}}$.

3.5.5 Analysis / Interpretation.

Since the parameters tested involve all of BOAST’s inputs except for prior strength, this experiment establishes that BOAST’s bias behavior is not predictable from inputs, and will be a factor of prior strength alone.

The lack of correlation between prediction bias and p_{fail} implies a lack of a statistically significant effect of p_{fail} on any form of bias, and the mean near 0.0 across all pass_ratio values implies no systemic bias in either direction across the parameter space.

Prediction bias, as the quality of the sample collected by BOAST, will be the primary driver of decision bias, as evidenced by the similar behavior of the two across the parameter space, and the matching of their standard deviations when using the raw distance $\text{pass_ratio} - p_{\text{fail, true}}$ for decision bias calculation.

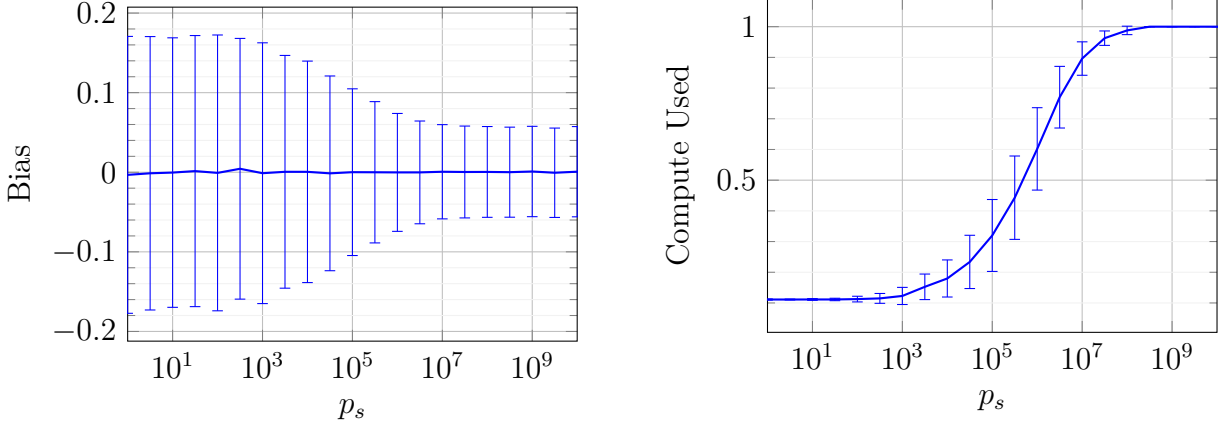
In addition, the very slight negative mean of decision bias across the parameter space may reflect a slight preference for A Type I error (false positive) over Type II error (false negative), which may be a desirable property in many applications, as it would lead to more conservative decisions. Due to the small magnitude of this bias, far below the standard deviation of the mean, it may not hold practical significance.

3.5.6 Summary / Recommendation.

These results indicate that prior strength is the key parameter influencing bias in BOAST, and should be the focus of further investigation.

It suggests that p_s is not a function of inputs, but a constant that can be pre-computed to achieve desired bias characteristics.

3.6 Experiment 6: Prior Strength vs Bias and Compute



(a) Bias vs increasing p_s

(b) $\frac{\text{iterations}}{k_{\text{initial}}}$ vs p_s

Figure 8: Comparison of bias behavior and convergence across p_s values

Objective. To investigate the relationship between prior strength, bias, and computational cost in BOAST.

3.6.1 Setup.

- We vary prior strength p_s from $10e0$ to $10e12$ in logarithmic increments of $10e0.5$.
- For each p_s , we perform 10,000 iterations of the following:
 - Draw a value x from a uniform distribution $U(0, 1)$.
 - Determine range for the rate of failure based on the following piecewise function:

$$\text{failure_rate} = \begin{cases} [0.001, 0.01] & x < 0.1 \\ [0.01, 0.05] & 0.1 \leq x < 0.3 \\ [0.05, 0.20] & 0.3 \leq x < 0.65 \\ [0.20, 0.50] & 0.65 \leq x < 0.9 \\ [0.50, 0.999] & x \geq 0.9 \end{cases}$$

- Draw a true failure probability $p_{\text{fail, true}}$ from a uniform distribution over the determined range.
- Run BOAST using the failure rate found, noting the pass_ratio value at the end of the run, the initial estimate for k and the number of iterations actually performed.
- Compute bias as $p_{\text{fail, true}} - \text{pass_ratio}$.
- Compute the fraction of initial k actually performed: $\frac{k_{\text{performed}}}{k_{\text{initial}}}$.

- Compute the mean and standard deviation of bias across all iterations.
- Compute the mean and standard deviation of fraction of initial k performed across all iterations.
- Repeat the above procedure 50 times, to get the mean and standard deviation of the above metrics across runs for each p_s value.

3.6.2 Definition of Metrics.

- $p_{\text{fail, true}}$: True failure probability drawn from the defined piecewise distribution, magnifying the presence of segments of the parameter space that are more likely to be encountered in practice, while still covering the full range of p_{fail} values.
- p_s : Prior strength parameter provided to BOAST.
- *bias*: Difference between true failure probability and observed pass ratio. The mean of bias represents systematic error, while the standard deviation represents variability in the error.
- *compute_fraction*: Fraction of initial k actually performed, representing computational cost.

3.6.3 Hypothesis.

We expect bias to remain centered around 0.0, but its standard deviation to decrease as prior strength p_s increases. We also expect computational cost to increase with prior strength, as higher p_s values lead to larger initial k estimates.

3.6.4 Results.

As shown in Figure 8b, the mean bias remains centered around 0.0 across all prior strength values tested, confirming part of our hypothesis.

However, while the standard deviation of bias does decrease with increasing prior strength, there is a ceiling effect observed around $p_s = 10e7$ beyond which further increases in prior strength yielded no additional reduction in bias variability, while still increasing computational cost.

3.6.5 Analysis / Interpretation.

There appears to be a sigmoidal saturation effect in bias standard deviation with respect to both bias standard deviation and fraction of worst-case compute performed.

This suggests there exists an optimal prior strength value that balances bias reduction and computational cost, beyond which further increases in prior strength yield diminishing returns in bias improvement while incurring higher computational costs.

3.6.6 Summary / Recommendation.

Further analysis is needed to precisely identify the optimal prior strength value that minimizes bias variability while controlling computational cost.

3.7 Experiment 7: Prior Strength Derivation

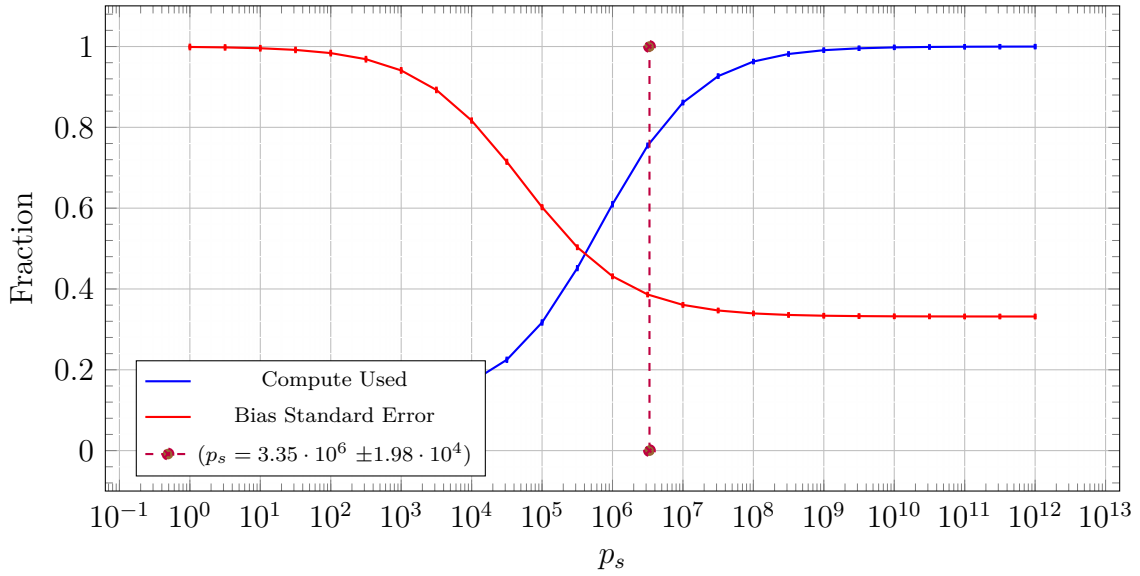


Figure 9: The compute / accuracy tradeoff in selecting p_s . Compute used is in the fraction of $k_{initial}$ actually performed in simulation, Bias is in the standard deviation of bias observed. Markers indicate recommended values for p_s .

Objective. To use the results from the experiment in Section 3.6 to derive a recommended prior strength value for BOAST based on desired bias characteristics. Based on the monotonic, sigmoidal behavior observed, we selected the hill equation as a suitable model.

3.7.1 Setup.

For the curve fits below, we will use the following hill curve function [2]:

$$y = y_{min} + (y_{max} - y_{min}) \frac{x^n}{k^n + x^n}$$

where:

Balance Factor	p_s	Compute	Bias	Δ Compute	Δ Bias	p_s Error
0.44	$7.08 \cdot 10^6$	0.83	$3.49 \cdot 10^{-2}$			$3.83 \cdot 10^4$
0.47	$4.71 \cdot 10^6$	0.8	$4.38 \cdot 10^{-2}$	$3.72 \cdot 10^{-2}$	$8.85 \cdot 10^{-3}$	$2.70 \cdot 10^4$
0.51	$3.35 \cdot 10^6$	0.76	$5.28 \cdot 10^{-2}$	$3.56 \cdot 10^{-2}$	$9.02 \cdot 10^{-3}$	$1.98 \cdot 10^4$
0.54	$2.48 \cdot 10^6$	0.73	$6.2 \cdot 10^{-2}$	$3.4 \cdot 10^{-2}$	$9.22 \cdot 10^{-3}$	$1.46 \cdot 10^4$
0.58	$1.9 \cdot 10^6$	0.69	$7.14 \cdot 10^{-2}$	$3.25 \cdot 10^{-2}$	$9.42 \cdot 10^{-3}$	$1.64 \cdot 10^4$
0.62	$1.49 \cdot 10^6$	0.66	$8.11 \cdot 10^{-2}$	$3.11 \cdot 10^{-2}$	$9.64 \cdot 10^{-3}$	$1.25 \cdot 10^4$
0.67	$1.19 \cdot 10^6$	0.63	$9.09 \cdot 10^{-2}$	$2.97 \cdot 10^{-2}$	$9.86 \cdot 10^{-3}$	$9.80 \cdot 10^3$
0.71	$9.7 \cdot 10^5$	0.61	0.1	$2.84 \cdot 10^{-2}$	$1.01 \cdot 10^{-2}$	$7.80 \cdot 10^3$
0.77	$7.97 \cdot 10^5$	0.58	0.11	$2.71 \cdot 10^{-2}$	$1.03 \cdot 10^{-2}$	$6.31 \cdot 10^3$
0.82	$6.62 \cdot 10^5$	0.55	0.12	$2.59 \cdot 10^{-2}$	$1.06 \cdot 10^{-2}$	$5.19 \cdot 10^3$

Figure 10: Table of optimal prior strength values for various balance factors β weighing bias reduction vs computational cost, based on hill curve fits to the normalized standard deviation of bias and normalized fraction of worst-case compute performed data.

- y : Normalized value at a given p_s .
- x : Prior strength p_s .
- y_{min} : Normalized value at $p_s = 1$.
- y_{max} : Normalized value at $p_s = 10^{12}$.
- k : Half-maximal effective concentration, representing the p_s value at which saturation is at 50%.
- n : Hill coefficient, describing the steepness of the curve.

We perform the following steps to derive the recommended prior strength value:

- Normalize the standard deviation of bias results to be a fraction of the maximum observed standard deviation across all prior strength values tested.
- Fit the hill curve function above to the normalized standard deviation of bias data using non-linear least squares optimization to find the best-fit.
- Fit the hill curve function above to the fraction of worst-case compute performed data using non-linear least squares optimization to find the best-fit.
- Calculate the r^2 value for each fit to assess goodness of fit.
- Introduce a balance factor β and a Lagrangian objective function [1] to weigh the importance of bias reduction vs computational cost:

$$\text{objective}_{\text{score}}(p_s) = \text{bias}_{\text{std}}(p_s) - \beta \cdot \text{compute}_{\text{frac}}(p_s)$$

- For a range of 200 β values in the logarithmic space from 10^{-5} to 10^1 , find the prior strength p_s that minimizes the objective function.
- Only consider solutions where p_s is in the observed range of 10^0 to 10^{12} .
- For each optimal p_s found, find the error on p_s using the delta method:

$$\text{error}(p_s) = \frac{\sigma \text{hill}_{\text{bias}}^2 + (\beta \cdot \sigma \text{hill}_{\text{compute}}^2)}{\frac{\partial \text{objective}(p_s, \beta)}{\partial p_s}}$$

- Build a table of optimal p_s values for each β with the following columns:
 - (1) β : Balance factor.
 - (2) p_s : Optimal prior strength for the given β .
 - (3) Normalized fraction of worst-case compute performed at optimal p_s .
 - (4) Normalized standard deviation of bias at optimal p_s .
 - (5) Delta between current and previous row's normalized fraction of worst-case compute performed.
 - (6) Delta between current and previous row's normalized standard deviation of bias.
 - (7) The error on p_s

3.7.2 Definition of Metrics.

- p_s : Prior strength parameter provided to BOAST.
- Normalized standard deviation of bias: Standard deviation of bias normalized to the maximum observed standard deviation across all prior strength values tested.
- Normalized fraction of worst-case compute performed: Fraction of worst-case compute performed normalized to the maximum observed fraction across all prior strength values tested.
- r^2 : Coefficient of determination for the hill curve fits, indicating goodness of fit.
- β : Balance factor weighing bias reduction vs computational cost.
- n, k : Hill curve parameters as defined above.
- $p_{s, \text{optimal}}$ Recommendation prior strength for a given β .
- $\text{error}(p_{s, \text{optimal}})$: Estimated error on the recommended prior strength.

3.7.3 Hypothesis.

Based on the monotonic, sigmoidal behavior and apparent saturation effects observed in Section 3.6, we expect the hill equation to provide a good fit to both the normalized standard deviation of bias and normalized fraction of worst-case compute performed data, with fair r^2 values indicating strong goodness of fit.

We also expect to find a fairly wide range of optimal prior strength p_s values corresponding to a flat region in bias, with the error on p_s estimates being moderate, contributing to a wider recommendation range.

3.7.4 Results.

The hill curve fits yielded high r^2 values:

- Normalized standard deviation of bias fit: $r^2 = 0.9989$
- Normalized fraction of worst-case compute performed fit: $r^2 = 0.9991$

For the curve of bias against p_s , we found the following:

$$y = 1.0 + (0.323 - 1.0) \frac{x^{0.548}}{(4.77 \cdot 10^4)^{0.548} + x^{0.548}}$$

For the curve of compute against p_s , we found the following:

$$y = 0.111 + (1.0 - 0.111) \frac{x^{0.639}}{(6.70 \cdot 10^5)^{0.639} + x^{0.639}}$$

The table of optimal prior strength p_s values for various balance factors β revealed a range of p_s values corresponding to a flat region in bias where further increases in prior strength yielded minimal bias improvement while incurring higher computational costs.

It also revealed small errors (most $< 0.5\%$) on the values of both fits, and by extension, on the recommended prior strength values.

The table shown in Figure 10 shows a value of $p_s = 3.35 \cdot 10^6 \pm 1.98 \cdot 10^4$ provides a good balance between bias reduction and computational cost, yielding a normalized standard deviation of bias of 0.052 and a normalized fraction of worst-case compute performed of 0.76.

3.7.5 Analysis / Interpretation.

Both r^2 exceeding 0.998 confirm the hill equation is an excellent model for both the normalized standard deviation of bias and normalized fraction of worst-case compute performed data. The tight errors on the fits further reinforce the reliability of the model.

As expected, n , or the steepness of the curve, was below 1 for both curves, indicating a gradual slope towards a saturation point, rather than a sharp cliff. The k values indicate that the bias curve reaches half of its maximum value at a much lower p_s than the compute curve, indicating that bias is more sensitive to changes in p_s at lower values, while compute continues to improve significantly as p_s increases. This results in a "window of efficiency" in which bias is relatively insensitive to changes in p_s , while compute continues to degrade significantly.

The table of optimal prior strength p_s values indicates a practical range for selecting p_s where the change in bias reduction becomes negligible compared to the increase in computational cost, allowing users to make informed decisions based on their specific bias tolerance and compute resource constraints.

The recommended value provides an option where increasing p_s to the maximum reasonable value yields 5% improvement in bias standard deviation at the cost of 24% increase in computational cost.

The $\beta = 0.51$ finding at the suggested p_s corresponds to a value determining the relative importance of bias reduction vs computational cost, that compute here is roughly half as important as bias reduction.

3.7.6 Summary / Recommendation.

We recommend a prior strength value of $p_s = 3.35 \cdot 10^6 \pm 1.98 \cdot 10^4$ for BOAST, balancing bias reduction and computational cost effectively for most applications.

The small errors on both the curve fits and recommended prior strength values provide confidence in the reliability of this recommendation.

3.8 Experiment 8: Quality of the Final Error Rate Estimate

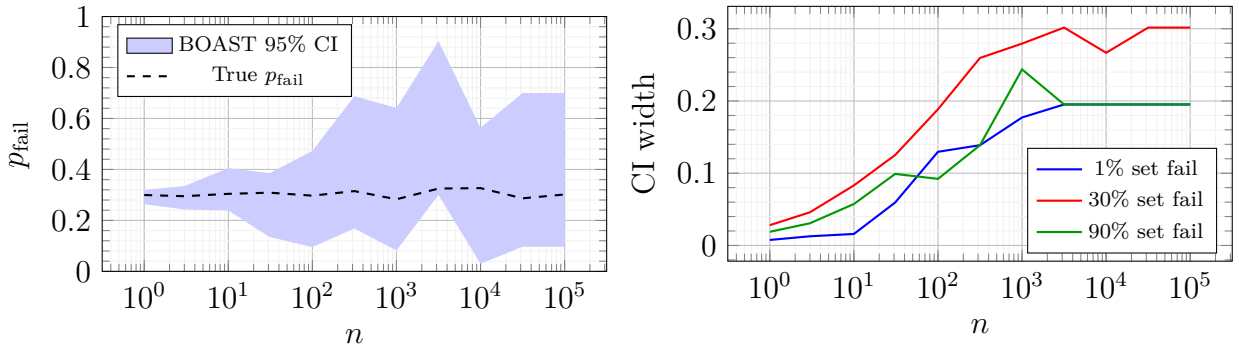


Figure 11: Graphs showing the true p_{fail} values against the reported confidence intervals from BOAST for various n (left) and the width of the confidence intervals against n (right).

Objective. To evaluate the quality of the final p_{fail} estimate provided by BOAST, by comparing the true p_{fail} values from simulations against the confidence intervals reported by BOAST across a range of dataset sizes n .

3.8.1 Setup.

We utilize a standard normal distribution $\mathcal{N}(0, 1)$ to generate synthetic datasets. A Z-test is applied as the pass/fail criterion, where a dataset of size n fails if the absolute Z-score $|Z| = |\frac{1}{\sqrt{n}} \sum x_i| > c$. The threshold c is analytically derived to target specific failure probabilities $p_{\text{fail}} \in \{0.01, 0.3, 0.9\}$.

For each n ranging from 10^0 to 10^5 , we simulate $100 \cdot k$ datasets (where k is BOAST’s sample size) to establish a ”ground truth” simulated failure rate. We then run BOAST with a 95% confidence target and compare its reported Wilson interval bounds against this ground truth.

3.8.2 Definition of Metrics.

- $p_{\text{fail, true}}$: The true failure probability obtained from the simulations for each dataset size n .
- $p_{\text{fail, boast, min}}$: The lower bound of the confidence interval for p_{fail} reported by BOAST.
- $p_{\text{fail, boast, max}}$: The upper bound of the confidence interval for p_{fail} reported by BOAST.
- Interval width: The width of the confidence interval reported by BOAST, calculated as $p_{\text{fail, boast, max}} - p_{\text{fail, boast, min}}$.

3.8.3 Hypothesis.

We hypothesize that the true failure probability $p_{\text{fail, true}}$ will consistently fall within the confidence intervals reported by BOAST across all tested dataset sizes n , and that the width of these confidence intervals will increase as n increases, reflecting a small initial estimated k .

3.8.4 Results.

We find that the true failure probability $p_{\text{fail, true}}$ indeed falls within the confidence intervals reported by BOAST for all tested dataset sizes n , confirming the reliability of BOAST’s estimates.

While the width of the confidence intervals increases with n , it appears to stabilize between $n = 10^3$ and $n = 10^4$, suggesting that BOAST’s sample size is sufficient to provide accurate estimates without excessive widening of the confidence intervals at larger dataset sizes.

3.8.5 Analysis / Interpretation.

The results confirm that BOAST provides accurate confidence intervals for p_{fail} estimates across a wide range of dataset sizes, with the true failure probability consistently falling within these intervals.

3.8.6 Summary / Recommendation.

BOAST’s final p_{fail} estimates are reliable and can be confidently used for decision-making, with the understanding that confidence interval widths may increase with larger dataset sizes but remain informative.

The effect can be mitigated with more conservative point-wise p values, which would lead to larger initial k estimates and thus more iterations performed by BOAST, improving the accuracy of the p_{fail} estimate and reducing confidence interval widths at larger dataset sizes.

4 Results

We evaluated the BOAST algorithm across a range of scenarios to assess its performance in terms of confidence accuracy, computational efficiency, and robustness to parameter mis-specification.

In the experiments described in Sections 3.1 and 3.2, we explored the effectiveness of BOAST’s initial failure probability estimation compared to simulated ground truth values. Figures 2 and 3 illustrate that BOAST’s estimates closely align with the actual failure probabilities, while remaining conservative, thus validating the algorithm’s reliability in practical applications.

In the subsequent experiment in Section 3.3, we tested the validity of the confidence interval BOAST uses to determine when to stop iterating. As shown in Figure 4, the lower-bound confidence intervals produced by BOAST consistently encompass the true failure probabilities across various dataset sizes and point-wise failure probabilities, demonstrating the robustness of the stopping criterion.

In addition, contrary to our hypothesis, the badly specified point-wise p values (blue and red curves) performed comparably to the well-specified values (green curves), indicating that BOAST is resilient to inaccuracies in initial parameter settings.

Next, in Section 3.4, we explored the effect of the damping constant on BOAST’s stopping condition via the floor on the number of iterations as set size increases. Figure 5 shows that the damping constant has a linear monotonic effect on the number of iterations performed, allowing for tuning based on available computational resources.

Finally, in Sections 3.5 through 3.7, we investigated the impact of prior strength on BOAST’s performance. Figure 9 shows that increasing prior strength leads to an improvement in bias of the failure probability estimates, albeit with diminishing returns, at the cost of increased

computational effort following the hill curve for both metrics.

These results collectively demonstrate that BOAST is an effective and adaptable algorithm for iterative outlier detection, capable of providing reliable estimates while allowing for customization based on specific application needs.

5 Discussion

The results presented in Section 4 highlight several key strengths of the BOAST algorithm, as well as areas for further exploration.

One notable finding is BOAST’s robustness to mis-specification of the point-wise failure probabilities. Contrary to our initial hypothesis, the performance of BOAST remained consistent regardless of whether the point-wise p values were well-specified or deliberately mis-estimated. This resilience suggests that BOAST can be effectively applied in scenarios where precise prior knowledge of failure probabilities is unavailable, enhancing its practical utility in a wider range of applications than initially anticipated.

Two possible explanations for this robustness are the conservative nature of the initial failure probability estimates due to the damping fraction model, as well as the effect of the relatively strong prior strength used in the Bayesian updating process (Section 3.7). Both factors likely contribute to stabilizing the estimates against inaccuracies in the initial parameters.

Another finding was the lack of inherent bias in BOAST’s failure probability estimates across various scenarios (See Section 3.5). Across the universe of input parameters tested, BOAST consistently produced a mean bias near zero, with very little variation. This characteristic is particularly advantageous in applications where unbiased estimates are critical, as it ensures that the algorithm does not systematically overestimate or underestimate failure probabilities regardless of specified parameters.

The introduction of the damping fraction to temper the number of iterations as set size increases also proved effective in controlling computational cost while maintaining reliable estimates. The model represents a saturating monotonic relationship with C modulating the effective binding strength between outliers in the dataset. This allows users to tune the algorithm based on available computational resources without significantly compromising accuracy. It is noteworthy that p_s and C should be considered a pair - modifying C will likely change the optimal p_s value.

During the update procedure, instead of a pure Bayesian update after each iteration, we employed a bulk update approach, aggregating evidence from multiple iterations before updating the posterior distribution on failure. This has the advantage of eliminating the possibility of artificially lengthy tests in the case of an absence of failure without compromising the Bayesian nature of the update.

Finally, the use of a frequentist stopping criterion based on confidence intervals proved effective in ensuring that BOAST halts iterations once sufficient evidence has been gathered.

The reported confidence intervals consistently encompassed the true failure probabilities, demonstrating the reliability of this stopping condition. This approach balances the need for computational efficiency with the requirement for accurate estimates, making it a practical choice for real-world applications.

5.1 Future Work

Future work could explore the extent of the independence assumption made in the BOAST model, particularly in datasets where outliers may exhibit correlations. Additionally, we investigated only the effects of normal outliers; investigating alternative prior distributions or adaptive prior strength mechanisms could potentially enhance the algorithm’s performance or applicability to a wider range of failure modes.

The presence in the derivation of the closed-form expression for the damping constant C of the cumulative hazard function, and the units in iterations per nat that it implies, suggests the possibility of a more rigorous theoretical analysis of the implications of C . By deriving our recommended value of $C = 1.864$ from the rule of three ($k \geq 3$), and Pareto-optimal thresholds ($q \geq 0.8$), C may be a heuristic threshold for the minimum required iterations per unit of uncertainty for many testing scenarios.

5.2 Example Applications

5.2.1 Documented Property Testing

Scenario A software library supports two functions, f and g , used for the same purpose but with different implementations. The library documentation states that f outperforms g for most inputs in a given metric. A tester wants to verify this claim by running randomized tests on both functions and comparing their performance.

Let $P = \{X^{(1)}, X^{(2)}, \dots\}$ be a countably infinite population of sets exhibiting the property in which f is documented to outperform g . Each set is of fixed size n .

Let:

$$F : \mathcal{X} \rightarrow \{0, 1\}, \quad F(X^{(j)}) = \begin{cases} 1 & \text{if } f \text{ outperforms } g \text{ on } X^{(j)} \\ 0 & \text{if } g \text{ outperforms or matches } f \text{ on } X^{(j)} \end{cases}$$

The tester’s goal is to determine a reasonable estimate of the proportion of sets in P for which f outperforms g , and to do so with a high degree of confidence while maintaining performance needed for CI pipelines. BOAST can be applied in this scenario to adaptively determine the number of iterations needed to reliably estimate the failure probability (i.e., the probability that g outperforms or matches f) and to provide a confidence interval on failure.

5.2.2 Performance Regression Testing

Scenario A software development team is implementing a new feature in their application, and they want to ensure that this addition does not degrade the performance of existing functionality while controlling for outliers in the performance measurements. They decide to use randomized testing to compare the performance of the application before and after the new feature is integrated.

Let $P = \{X^{(1)}, X^{(2)}, \dots\}$ be a countably infinite population of sets representing inputs to the application, where each set is of fixed size n .

Let:

$$F : \mathcal{X} \rightarrow \{0, 1\}, \quad F(X^{(j)}) = \begin{cases} 1 & \text{if performance is acceptable on } X^{(j)} \\ 0 & \text{if performance degrades on } X^{(j)} \end{cases}$$

The team's goal is to estimate the probability of performance degradation due to the new feature, while accounting for the presence of outliers in the performance data. BOAST can be applied here to adaptively determine the number of iterations needed to reliably estimate this probability and to provide confidence intervals, ensuring that the testing process is both efficient and informative for decision-making regarding the new feature's integration.

5.2.3 Robustness Testing

Scenario A function must be tested to meet certain robustness criteria, such as handling edge cases or extreme inputs, but which is known and expected to fail on a small proportion of inputs. The goal is to estimate the failure probability of the function under these conditions, since distribution of the synthetic test data is well understood.

Let $P = \{X^{(1)}, X^{(2)}, \dots\}$ be a countably infinite population of sets representing inputs to the function, where each set is of fixed size n .

Let:

$$F : \mathcal{X} \rightarrow \{0, 1\}, \quad F(X^{(j)}) = \begin{cases} 1 & \text{if the function performs robustly on } X^{(j)} \\ 0 & \text{if the function fails on } X^{(j)} \end{cases}$$

The tester's goal is to estimate the probability of failure under these conditions, and assert that it is below a certain threshold with high confidence. BOAST can be applied to adaptively determine the number of iterations needed to reliably estimate this failure probability, to provide confidence intervals, and to test the assertion that the failure probability is below the specified threshold, ensuring that the function meets the required robustness criteria.

6 Conclusion

We have presented the Bayesian Outlier-Aware Sequential Testing (BOAST) algorithm, a method for adaptively determining the number of iterations required to achieve a desired

confidence level in randomized testing. By combining Bayesian statistics with iterative refinement based on observed failure rates, BOAST improves upon traditional fixed-iteration approaches.

Empirical evaluation shows that BOAST is robust to poorly specified parameters and accurately models failure rates under independent, normally distributed errors. It balances computational cost with confidence in results, making it practical for resource-constrained settings such as CI pipelines. Future work could explore extending BOAST to correlated or non-normal error models, further broadening its applicability.

References

- [1] Hugh Everett. Generalized lagrange multiplier method for solving problems of optimum allocation of resources. *Oper. Res.*, 11(3):399–417, June 1963.
- [2] Rudolf Gesztelyi, Judit Zsuga, Balázs Varga, Bela Juhasz, and Arpad Tosaki. The hill equation and the origin of quantitative pharmacology. *Archive for History of Exact Sciences*, 66, 07 2012.
- [3] Chia-Yu Hsu and Shubhanshu Shekhar. Classifier-based nonparametric sequential hypothesis testing, 2026.
- [4] Sampurna Kundu, Jayant Jha, and Subir Kumar Bhandari. An efficient adaptive sequential procedure for simple hypotheses with expression for finite number of applications of less effective treatment, 2025.
- [5] Glenford J. Myers, Corey Sandler, and Tom Badgett. *The Art of Software Testing*. Wiley, 3 edition, 2011.
- [6] Kizito Salako and Rabi Muhammad. Conservative software reliability assessments using collections of bayesian inference problems, 2025.
- [7] A. Wald. Sequential tests of statistical hypotheses. *The Annals of Mathematical Statistics*, 16(2):117–186, 1945.
- [8] Edwin B. Wilson. Probable inference, the law of succession, and statistical inference. *Journal of the American Statistical Association*, 22(158):209–212, 1927.

Appendix

Code

Rust Implementation

The repository for this paper contains a Rust implementation of the BOAST algorithm used for the experiments described. The code can be found at: <https://github.com/rscarson/boast/>

It is a ready-to-use implementation in the form of a crate named `boast`, which can be included in other Rust projects via Cargo.

Python Implementation

This code and a sample run can be found at: <https://github.com/rscarson/boast/blob/master/boast.py>

```
1 import math
2 import random
3 import sys
4 import time
5
6 DAMPING_CONSTANT = 1.864
7 PRIOR_STRENGTH = 3.35e6
8
9 def p_fail_interval(pass_ratio, k, z):
10     p = 1 - pass_ratio
11     denom = 1.0 + (z * z) / k
12
13     center = (p + (z * z) / (2.0 * k)) / denom
14     half_width = (
15         z * math.sqrt((p * (1.0 - p) / k) + (z * z) / (4.0 * k * k))
16         / denom
17     )
18
19     p_fail_lower = max(0.0, center - half_width)
20     p_fail_upper = min(1.0, center + half_width)
21
22     return p_fail_lower, p_fail_upper
23
24 def boast_k(q, p_fail):
25     """Calculates the required number of iterations k for BOAST given
26     confidence q and failure probability p_fail."""
27     return math.ceil(abs(math.log(1.0 - q) / math.log(1.0 - p_fail)))
```

```

28 def boast_run(data, f_transform, f_test, q, p, pass_ratio, timeout_s =
None, C = DAMPING_CONSTANT, p_s = PRIOR_STRENGTH):
29     """
30     Runs the BOAST algorithm on the given data.
31
32     Parameters:
33         - data: The dataset to run BOAST on.
34         - f_transform: Function to transform a set = fn(dataset, seed)
        -> transformed_dataset
35         - f_test: Function to test a set = fn(transformed_dataset) ->
        bool (pass/fail)
36         - q: Desired confidence level (e.g., 0.95 for 95% confidence)
37         - p: Pointwise failure probability estimate (should
        underestimate true failure probability)
38         - pass_ratio: Required pass ratio to consider the test
        successful (e.g., 0.99 for 99% pass rate)
39         - timeout: Optional timeout for the test function, in seconds (
        default: None)
40         - C: Damping constant (default: 1.864)
41         - p_s: Prior strength (default: 3.35e6)
42     """
43     start_time = time.monotonic()
44
45     # Calculate damping fraction and adjusted setwise failure
        probability
46     n = len(data)
47     damping_fraction = 1.0 / (1.0 + (C * n * p))
48     p_prime = p * damping_fraction
49     p_fail = 1.0 - (1.0 - p_prime)**n
50
51     # Calculate initial k
52     k = boast_k(q, p_fail)
53     k_initial = k
54
55     # Initialize priors
56     alpha = p_s * p_fail
57     beta = p_s * (1.0 - p_fail)
58
59     print(
60         f"Starting with initial pass count k = {k_initial}, "
61         f"confidence = {q * 100:.2f}%, "
62         f"required pass ratio = {pass_ratio * 100:.2f}%"
63     )
64
65     print(f"\rFinished 0 / {k} iterations. 0 failures reported.", end='
')
66     sys.stdout.flush()

```

```

67
68 iterations = 0
69 passes = 0
70 unreported_passes = 0
71 last_failing_seed = 0
72 while iterations < k:
73     if iterations - passes > 0 and pass_ratio >= 1.0:
74         break
75
76     if timeout_s is not None and (time.monotonic() - start_time) >
77         timeout_s:
78         print("\nTimeout reached, terminating BOAST run.")
79         break
80
81     print(f"\rFinished {iterations} / {k} iterations. {iterations -
82         passes} failures reported.", end='')
83     sys.stdout.flush()
84
85     # Perform transformation and test
86     seed = random.getrandbits(64)
87     transformed_data = f_transform(data, seed)
88     passed = f_test(transformed_data)
89
90     # Record result
91     iterations += 1
92     if passed:
93         passes += 1
94         unreported_passes += 1
95     elif passes < iterations:
96         # Failure detected, update priors and recalculate k
97         alpha += 1
98         beta += unreported_passes
99         unreported_passes = 0
100
101         last_failing_seed = seed
102         p_fail = alpha / (alpha + beta)
103         k = boast_k(q, p_fail)
104
105     print(f"\rFinished {iterations} / {k} iterations. {iterations -
106         passes} failures reported.\n")
107
108     #
109     # Final readout
110     #
111     failures = iterations - passes
112     ratio = (passes / iterations)

```

```

111 (p_fail_lower, p_fail_upper) = p_fail_interval(ratio, k, 1.96)
112 print(
113     f"\nWith 95% confidence, the true failure rate is between {
        p_fail_lower * 100.0:.2f}% and {p_fail_upper * 100.0:.2f}%."
114     ,
115 )
116 print(f"{failures}/{iterations} tests failed ({ratio * 100.0:.2f}%
        pass)")
117
118 if iterations < k:
119     print(f"BOAST terminated early after {iterations} iterations -
        not enough evidence to reach required confidence.")
120     return False
121
122 if ratio < pass_ratio:
123     print(f"BOAST failed to meet required pass ratio of {pass_ratio
        * 100:.2f}%.")
124     print(f"Observed pass ratio: {ratio * 100:.2f}%.")
125     print(f"Last failing seed: {last_failing_seed}.")
126     return False
127
128 print("BOAST passed successfully.")
129 return True

```

Figure and Table Listings

1	Flowchart of the BOAST testing procedure.	7
2	Graph showing the conservative estimates of p_{fail} vs simulated true p_{fail} for various dataset sizes n	16
3	Graph showing the predicted number of iterations k to observed failure vs the simulated results	18
4	Graph showing the mean and standard deviation of lower-bound confidence intervals on the sample of iterations observed by BOAST from a simulation of 10,000 trials, compared to the simulated true p_{fail} for various dataset sizes n , and for various values of point-wise p . Green was deliberately underestimated, blue was exact, and red was deliberately overestimated. Markers indicate the number of iterations k performed by BOAST at each point.	20
5	Left: Graph of k_{floor} vs C values in the defined parameter space. Right: Table showing values of k_{floor} for $C_{\text{mean}} \pm C_{\sigma}$ for various confidence levels q	22
6	Graph demonstrating the lack of correlation between p_{fail} , pass_ratio , and bias in the base case of $p_s = 1$	24
7	Scatter plot showing the distribution of true p_{fail} values across all iterations of the experiment, demonstrating the coverage of the parameter space.	25
8	Comparison of bias behavior and convergence across p_s values	28
9	The compute / accuracy tradeoff in selecting p_s . Compute used is in the fraction of k_{initial} actually performed in simulation, Bias is in the standard deviation of bias observed. Markers indicate recommended values for p_s	30
10	Table of optimal prior strength values for various balance factors β weighing bias reduction vs computational cost, based on hill curve fits to the normalized standard deviation of bias and normalized fraction of worst-case compute performed data.	31
11	Graphs showing the true p_{fail} values against the reported confidence intervals from BOAST for various n (left) and the width of the confidence intervals against n (right).	34